

**Gagnaskipan spring 2025 - exam 3** Full name: \_\_\_\_\_

Mark the correct answers to the multiple-choice questions. There can only ever be one right answer.

3%

What is the time complexity of operations in a binary tree heap, where  $n$  is the total number of items/nodes in the tree?

- ☒ add:  $O(\log n)$ , remove:  $O(\log n)$
- ☐ add:  $O(\log n)$ , remove:  $O(1)$
- ☐ add:  $O(n)$ , remove:  $O(\log n)$
- ☐ add:  $O(n)$ , remove:  $O(1)$

3%

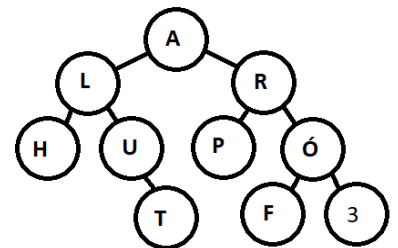
If data is entered into a BST in a sequence ordered by key, what will the time complexity of finding a value in the resulting tree be?:

- ☒  $O(n)$
- ☐  $O(n^2)$
- ☐  $O(\log n)$
- ☐  $O(1)$

3%

In which order are the nodes visited if the tree in the **image** is traversed postorder?

- ☒ H T U L P F 3 Ó R A
- ☐ \_ H U T L P F Ó 3 R A
- ☐ \_ H T U L A P F 3 Ó R
- ☐ \_ T H U L A F 3 Ó P R



3%

What is the time complexity of removing in a balanced Binary Search Tree?

- ☐  $O(n)$
- ☐  $O(n^2)$
- ☒  $O(\log n)$
- ☐  $O(1)$

3%

What is the time complexity of finding in a Hashmap that has a good hash value distribution

- ☐  $O(\log n)$
- ☐  $O(n)$
- ☐  $O(n^2)$
- ☒  $O(1)$

3%

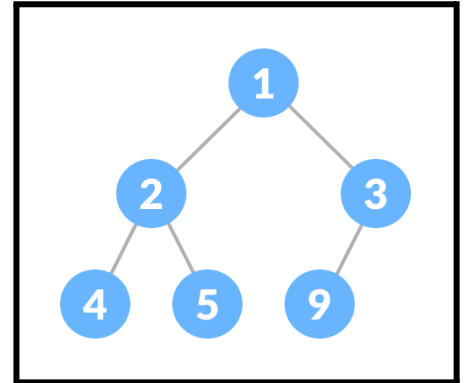
What is the time complexity of printing all elements in a Binary Search Tree using preorder?

- ☐  $O(n^2)$
- ☐  $O(\log n)$
- ☒  $O(n)$
- ☐  $O(1)$

3%

Which value from the heap in the **image** will be returned when we call the function: `remove()`

- ☐ 3
- ☐ 9
- ☐ 2
- ☒ 1



3%

What is the rule that must be kept in all nodes in a Min Heap?

- ☐ Everything left of a node is smaller and everything right of a node is larger
- ☒ The parent of a node is smaller and the children are larger
- ☐ All nodes in a specific layer must be smaller than nodes in the below layer
- ☐ None of the other options are True

3%

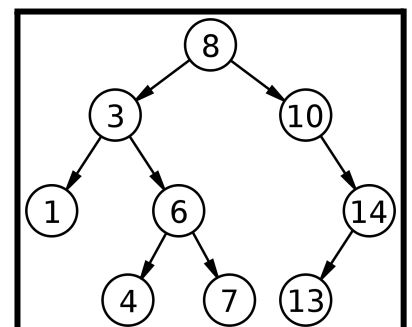
If data is entered into a Hashmap using a hash function that returns the same hash for all key values, what will the time complexity of finding a value in the resulting Hashmap be?

- ☒  $O(n)$
- ☐  $O(n^2)$
- ☐  $O(\log n)$
- ☐  $O(1)$

3%

What will the output be if we print the tree in the **image** using preorder?

- ☐ 1 3 4 6 7 8 10 13 14
- ☐ 6 4 7 3 1 8 10 13 14
- ☐ 1 4 7 6 3 13 14 10 8
- ☒ 8 3 1 6 4 7 10 14 13



#### 40% BSTMap

You are given a definition of the class BSTMap and the class BSTNode.  
Finish implementing the functions below.

- 10% insert(key, data)
  - Adds the key and data to the Map
    - If key already exists do nothing
- 10% find(key)
  - Returns the data associated with the key parameter
    - If key doesn't exist in the collection return None instead
- 10% update(key, data)
  - Sets the data value of the value pair with equal key to data
    - If key doesn't exist in the collection do nothing
- 10% count\_odd\_elements()
  - Returns an integer value, the amount of nodes in the tree containing **keys** that are odd numbers. You can assume that all keys are integers.

class BSTNode:

```
def __init__(self, key=None, data=None, left=None, right=None):
    self.key = key
    self.data = data
    self.left = left
    self.right = right
```

class BSTMap:

```
def __init__(self):
    self.root = None
    self.size = 0
```

```
def insert(self, key, data):
    #your code here, helper functions are encouraged
```

```
def find(self, key):  
    #your code here, helper functions are encouraged
```

```
def update(self, key, data):  
    #your code here, helper functions are encouraged
```

```
def count_odd_elements (self):  
    return self.count_odd_elements_recur(self.root)  
  
def count_odd_elements_recur(self, node):  
    if node == None:  
        return 0  
    count = 0  
    if node.key % 2 == 1:  
        count += 1  
    return count + self.count_odd_elements_recur(node.left) +  
self.count_odd_elements_recur(node.right)
```

### 30% HashMap

You are given a definition of the class HashMap.

**IMPORTANT:** You are also “given” a **Bucket** class that you should use as the internal Map structure. The implementation of the functions in the Bucket class should not matter.

You have access to the following functions in the **Bucket** class:

insert(key,data), contains(key), find(key), remove(key), update(key, data),  
\_\_setitem\_\_(key,data), \_\_getitem\_\_(key)

Finish implementing the functionality of these operations:

- 10% remove(key)
  - Removes the value pair with equal key from the collection
    - If key doesn't exist in the collection, do nothing
- 10% contains(key)
  - Returns True if equal key is found in the collection, otherwise False
- 10% find(key)
  - Returns the data associated with the key parameter
    - If key doesn't exist in the collection return None instead

class HashMap:

```
def __init__(self):
    self.capacity = 8
    self.size = 0
    self.bucket_arr = [None] * self.capacity
    for i in range(self.capacity):
        self.bucket_arr[i] = Bucket()

def remove(self, key):
    #your code here
```

```
def contains(self, key):  
    #your code here
```

```
def find(self, key):  
    #your code here
```